

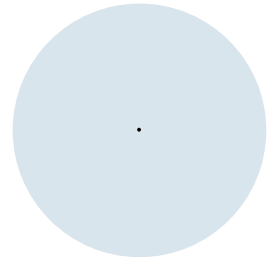
Einführung in die Programmierung (WIN+BIT)

Dateien, Mehrdimensionale Arrays

Prof. Dr. Johannes C. Schneider / Prof. Dr. Markus Eiglsperger



Dateizugriff



Dateizugriff

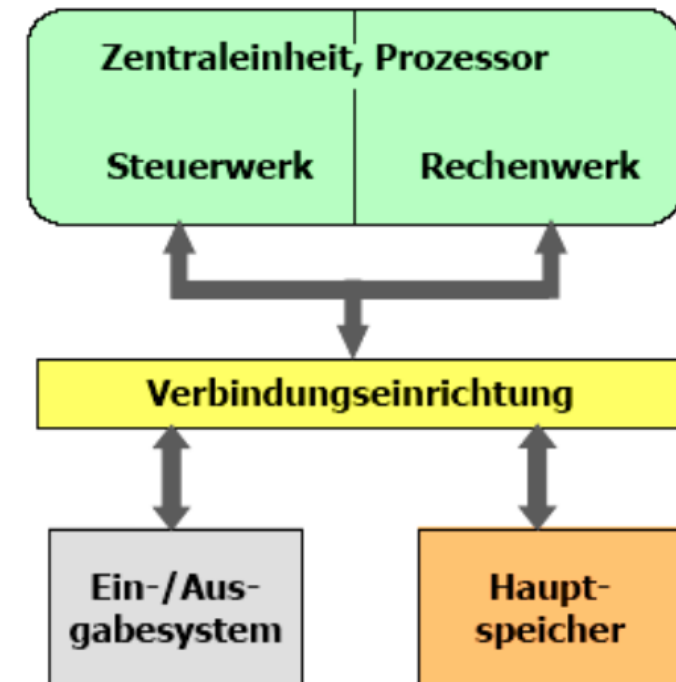
- Daten aus Dateien lesen und in Dateien schreiben
- Sinnvoll für große Datenmengen
- Ähnlich wie Einlesen von Eingaben über Tastatur (`java.util.Scanner`)

4. Grundlagen der Rechnerarchitektur

Komponenten eines von Neumann Rechners

▪ Ein-/ Ausgabesystem (Peripheriegeräte)

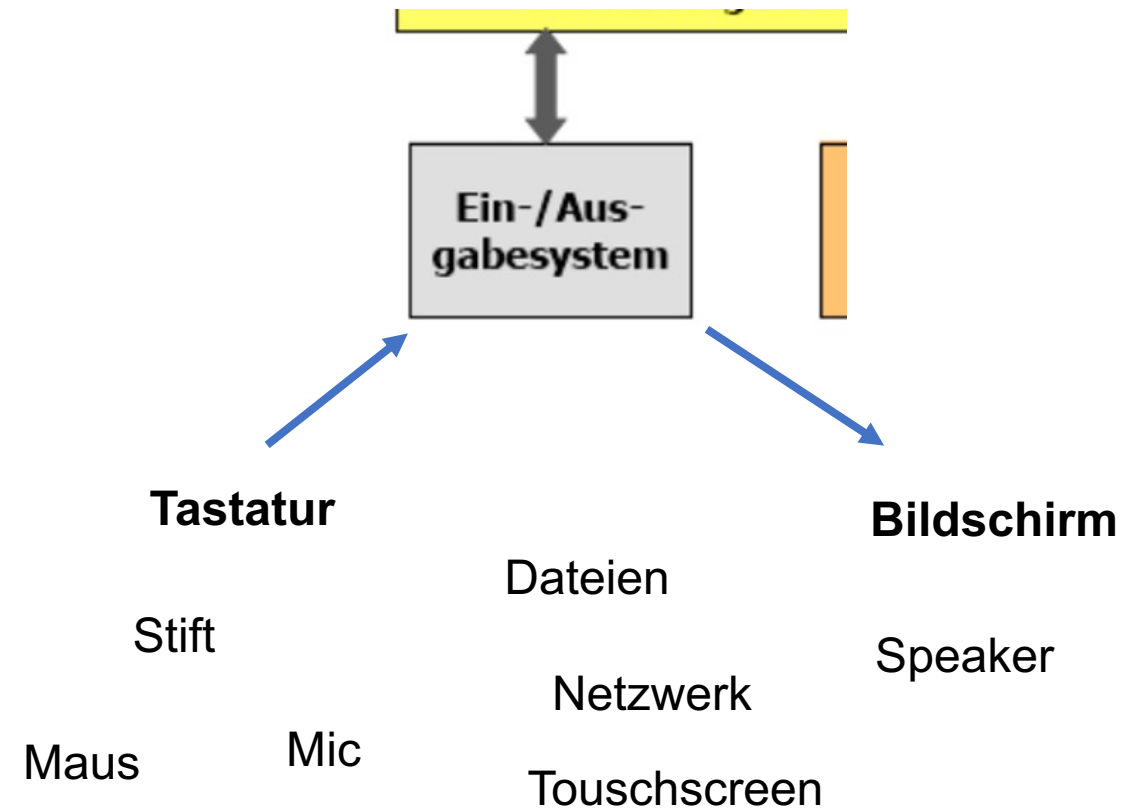
- Geräte zur Eingabe von Daten und Programmen und zur Ausgabe der verarbeiteten Daten (Bildschirm, Drucker, Terminals, ...)
- Diese Geräte sind über Ein-/Ausgabe-Schnittstellen mit dem Rechner verbunden.
- Die Verbindung der Schnittstellen mit dem Prozessor (und zu den Peripheriegeräten) geschieht durch Adress-, Daten- und Steuerleitung.



Ein-/Ausgabe in Java

Standard In / Standard Out

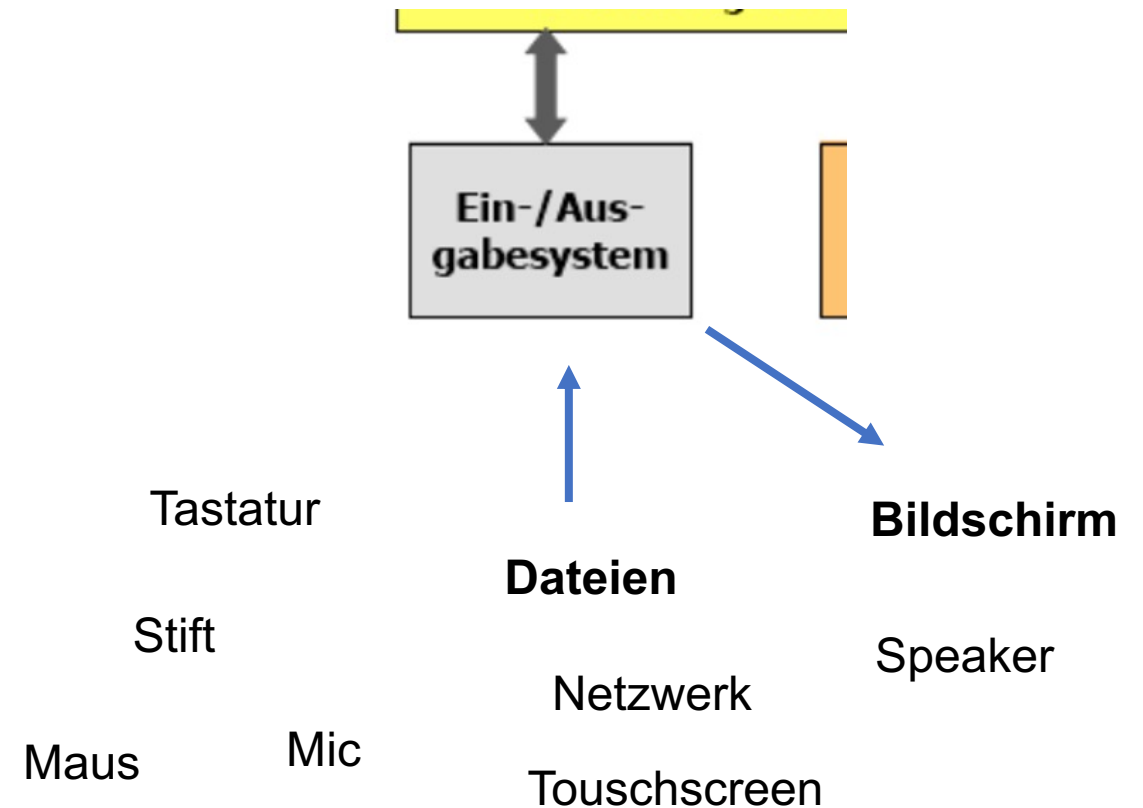
- Jedes Programm hat einen Standard-Eingabe Kanal und einen Standard Ausgabe Kanal für Text Eingabe / Ausgabe.
 - Standard-In (**System.in**) verweist auf das Peripheriegerät, dass als **Eingabegerät** angenommen wird.
Dies ist typischerweise die Tastatur
 - Standard-Out (**System.out**) verweist auf das Peripheriegerät, dass als **Ausgabegerät** angenommen wird.
Dies ist typischerweise ein Textfenster (Console) auf dem Bildschirm.



Ein-/Ausgabe in Java

Standard In / Standard Out

- Die Standardeingabe kann auch einer anderen Eingabequelle zugeordnet werden, z.B. **einer Datei**.



Einlesen einer CSV-Datei (Zeiterfassung)

- Wir lesen eine Datei `oktober2025.csv` mit Datum, Soll-Stunden und Ist-Stunden ein und berechnen
 - Gesamt-Sollstunden
 - Gesamt-Iststunden
 - Über-/Minusstunden
 - Gesamtlohn
- Fahrplan:
 1. Path, relativer und absoluter Pfad, existiert Datei? Dabei beachten: Imports und `IOException`
 2. Datei zeilenweise einlesen mit Hilfe von Scanner, Inhalt mit Zeilennummern ausgeben
 3. Zeilen zerlegen: `String.split()`
 4. Summiere soll und ist Stunden

Einlesen einer CSV-Datei (Komponisten)

- Wir lesen eine Datei `composers.csv` mit Geburtsjahr, Todesjahr und Namen von verschiedenen Komponist:innen ein und berechnen das Alter der Komponist:innen.
- Fahrplan:
 1. Path, relativer und absoluter Pfad, existiert Datei? Dabei beachten: Imports und `IOException`
 2. Datei zeilenweise einlesen mit Hilfe von Scanner, Inhalt mit Zeilennummern ausgeben
 3. Zeilen zerlegen: mit `String.split()`
 4. (Ungefähres) Alter berechnen
- Das alles folgt (zur Nachbereitung) auch noch mal auf den folgenden Folien als Einzelschritte (und verschiedene Versionen von `ReadingFiles1` und `ReadingFiles2`).
- Alternative: Zeile mit einem eigenen Scanner verarbeiten.

Java-Standardbibliothek

- Seit Java 7 "neue" API für Dateizugriff in Package [java.nio.file](#)
- [Path](#): *An object that may be used to locate a file in a file system.*
- Erzeugung von Path über [Paths.get\(<FileName>\)](#), wobei <FileName> absolut oder relativ sein kann
- Hilfsmethoden in [Files](#), z.B. [Files.exists\(Path\)](#): `boolean`
- Außerdem: [java.util.Scanner](#) zum "Zerlegen" des Inhalts der Datei

Hinweise:

- Dateizugriff kann fehlschlagen. Methoden werden dann eine *Exception*. Erkennbar durch Compiler-Fehler "Unhandled exception: <Exception>".
- In diesem Fall einfach hinter die main-Methode "throws <Exception> schreiben, mehrere mit Komma getrennt (s. IOException im Code-Beispiel).
- Exceptions werden später genauer behandelt.

Absolute Pfade, Trennzeichen

- Pfade, die mit dem Laufwerksbuchstaben oder mit / beginnen, nennt man **absolute Pfade**.
- Pfade werden unter Windows mit \ getrennt und unter MacOS und Linux mit /, Beispiele:
 - C:\Users\Chris\Documents\Thesis.docx (Windows)
 - /Users/Chris/Documents/Thesis.docx (MacOS)
 - /home/chris/Documents/Thesis.docx (Linux)
- Achtung: \ muss in einem Java-String escapt werden:
"C:\\Users\\Chris\\Documents\\Thesis.docx"
- Java wandelt unter Windows das Trennzeichen automatisch um, daher kann man auch unter Windows diese Pfadangabe verwenden: "C:/Users/Chris/Documents/Thesis.docx"
- **Tipp:** Verwenden Sie immer / als Trennzeichen, dann funktionieren die Pfade unabhängig vom Betriebssystem

Relativer Pfad

- `Paths.get` interpretiert Pfade, die nicht mit dem Laufwerk oder / beginnen, **relativ** zu dem Verzeichnis, aus dem heraus Java ausgeführt wird
 - `Paths.get(".")`: aktuelles Verzeichnis, in dem das IntelliJ-Projekt liegt, z.B. `C:\Users\Chris\EPR\Area`
 - `Paths.get("datei.txt")`: z.B. `C:\Users\Chris\EPR\Area\datei.txt`
 - `Paths.get("src/HelloWorld.java")`: z.B. `C:\Users\Chris\EPR\Area\src\HelloWorld.java`
- `Paths.get(".").toAbsolutePath()` wandelt den Pfad in einen absoluten Pfad um
- `System.out.println(Paths.get("src/HelloWorld.java").toAbsolutePath())`
// Ausgabe ähnlich wie `C:\Users\Chris\EPR\Area\src\HelloWorld.java`
- **Tipp:** Verwenden Sie in Ihrem Source-Code keine absoluten Pfade, weil diese ggf. nur auf Ihrem Rechner funktionieren.
 - `Paths.get("C:\\Users\\Chris\\EPR\\Area\\data\\plz.csv")`
 - besser: `Paths.get("data/plz.csv")`

Dateizugriff (Lesen)

- Mit Scanner kann man eine Datei Zeile für Zeile einlesen.

```
Path file = Paths.get("./composers.csv");
Scanner fileScanner = new Scanner(file, "UTF-8");
// read line by line
while (fileScanner.hasNextLine()) {
    String line = fileScanner.nextLine();
    // parse line, see ReadingFiles3
}

fileScanner.close();
```

Man muss das passende Encoding wählen (sonst werden z.B. Umlaute falsch dargestellt). Seit Java Version 18 ist UTF-8 der Standard (und man kann das an dieser Stelle auch weglassen)



code10.
Reading
Files1-3

Alternative: Ganze Datei als String einlesen

```
Path composersFile = Paths.get("./composers.csv");  
String content = Files.readString(composersFile, StandardCharsets.UTF_8);  
System.out.println(content);
```

- Aufpassen bei großen Dateien (kompletter Inhalt landet im Arbeitsspeicher)



code10.
ReadingFile
ToString

Dateizugriff (Schreiben)

```
Path outputFile = Paths.get("./output.txt");
try (PrintWriter writer = new PrintWriter(outputFile.toFile(), "UTF-8")) {
    writer.print("Hello ");
    writer.println("World!");
    for (int i = 0; i < 10; i++) {
        writer.printf("%d² = %d\n", i, i * i);
    }
}
```

- Schreiben von Strings über PrintWriter
- bietet dieselben Methoden wie System.out
- Daten werden zeichen-/zeilenweiser geschrieben
- man kann so auch große Dateien produzieren, ohne alle Daten vorher im Speicher halten zu müssen

Dateizugriff (Schreiben)

```
Path outputFile = Paths.get("./output.txt");
try (PrintWriter writer = new PrintWriter(outputFile.toFile(), "UTF-8")) {
    writer.print("Hello ");      Path wird in altes File-Objekt umgewandelt,
    writer.println("World!");    was von Printwriter erwartet wird
    for (int i = 0; i < 10; i++) {
        writer.printf("%d² = %d\n", i, i * i);
    }
}
```

Spezieller Block (try-with-resources), der dafür sorgt, dass die Dateiressourcen auch im Fehlerfall geschlossen werden



code15.
Writing
Files

Alternative: String in Datei schreiben

```
String output = "first line\nsecond line";  
Path path = Paths.get("./output.txt");  
Files.writeString(path, output);
```

- Sehr einfache und praktische Methode
- Man kann den String sukzessive länger machen (z.B. in einer Schleife) über `output += "noch mehr Inhalt.."`
- Aufpassen bei großen Datenmengen, weil diese in den Arbeitsspeicher des Rechners passen müssen.



code10.
Writing
StringToFile

Beispiel: Kalender

- Kalenderdaten gehen nach Beendigung des Programmes verloren.
- Bieten speichern / laden Option für den Benutzer an.
- Benutzer muss Dateinamen eingeben
- Neue Funktionalität wird in separaten Methoden implementiert.



Inhalt anhängen

- Manchmal möchte man den bestehenden Inhalt einer Datei beibehalten und weiteren Inhalt an das Ende einer Datei anhängen
 - z.B. Log-Ausgabe eines Programmes
- Dies muss man entsprechend einstellen, siehe:



code10.

WritingFiles
Append



code10.

WritingString
ToFileAppend

Weitere Dateioperationen

Link zur genauen Dokumentation

- Die Utility-Klasse `java.nio.file.Files` enthält viele nützliche Funktionen:

```
Path output = Paths.get("output.txt");
Path output2 = Paths.get("output2.txt");
System.out.println(Files.exists(output));
System.out.println(Files.exists(output2));
Files.copy(output, output2);
Files.move(output, output2, StandardCopyOption.REPLACE_EXISTING);
Files.delete(output2);
Files.deleteIfExists(Paths.get("somefile.txt"));
Files.createDirectory(Paths.get("subdirectory"));
Files.createDirectories(Paths.get("subdirectory/sub"));
```

optionaler dritter Parameter, um vorhandene Datei zu überschreiben. Funktioniert auch bei copy

- Aufgepasst: Manche Methoden werfen eine `IOException` (mehr dazu später). Man muss daher die `main`-Methode anpassen:

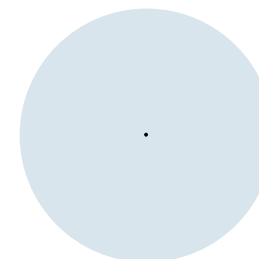
```
public static void main(String[] args) throws IOException { ... }
```



code10.
File

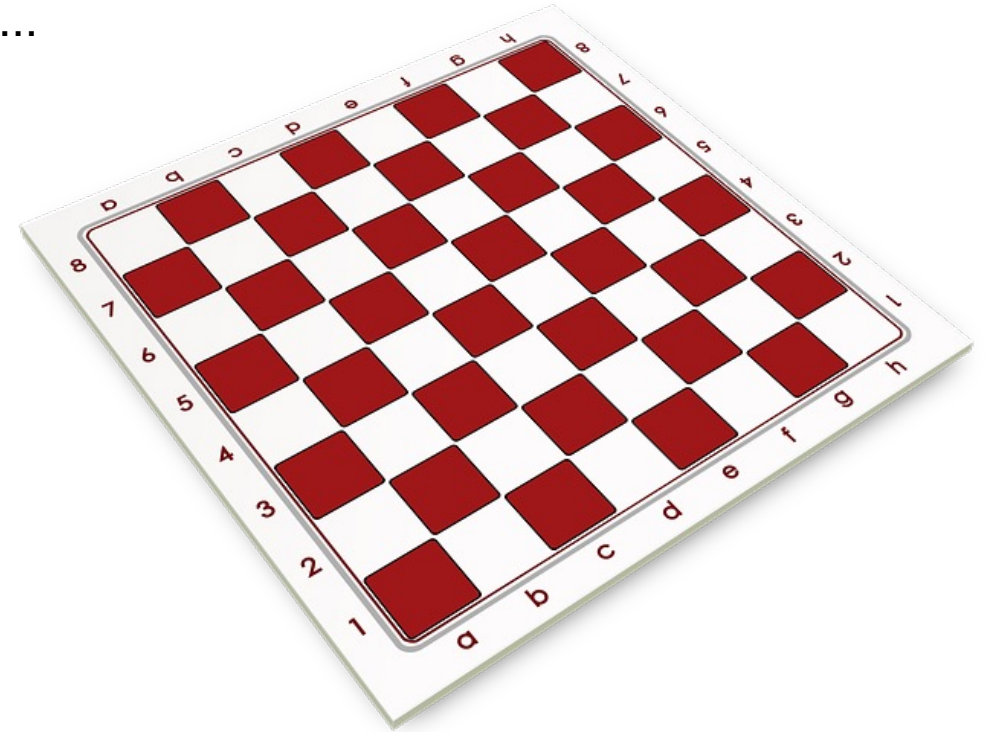
Management

Mehrdimensionale Arrays



Mehrdimensionale Arrays

- Beispiele
 - Kalender für mehrere Monate/Tage/Stunden
 - Schachbrett, Spielfeld, Planquadrate, Tariftabellen, ...
 - Würfel, Raummodelle, ...



Array-Initializer, zweidimensional

```
String[][] teams = {  
    {"Peter", "Paul"},  
    {"Anna", "Beate", "Emma"},  
    {"Thomas", "Sabrina"}  
};
```

	0	1	2
0	"Peter"	"Paul"	
1	"Anna"	"Beate"	"Emma"
2	"Thomas"	"Sabrina"	

- Peter steht in 0. Zeile und 0. Spalte `teams[0][0]`
- Emma steht in 1. Zeile und 2. Spalte `teams[1][2]`
- "Array von Arrays":
 - team enthält drei Arrays, da `teams.length` 3 ist.
 - `teams[0]` ist selbst ein (eindimensionales) Array mit den beiden Einträgen "Peter" und "Paul" und `teams[0].length` gleich 2
- Dieses Array ist nicht "rechteckig", da die einzelnen Zeilen unterschiedliche viele Spalten haben.

jshe11>



code11.
Multidimen-
sionalArray
Basics

Kalender erweitern

vorher: nur eine Zeile/Dimension,
hier umgebrochen dargestellt:

Arrays

- Zur Motivation: Ein Kalender
- Viele wiederkehrende Fälle
- Bsp:
 - Terminverwaltung
 - Zuteilung



© 2011
W 1
S 1 Einführung in die Programmierung (WIN+BIT), Prof. Dr. Johannes C. Schneider 49

	00:00	01:00	02:00	03:00	04:00	05:00	06:00	07:00	08:00	09:00	...	23:00
Tag 1											...	
Tag 2												
Tag 3												
Tag 4												
Tag 5												
Tag 6											...	
Tag 7											...	
Tag 8											...	
:	:	:	:	:	:	:	:	:	:	:		:
Tag 31											...	

Zweidimensionale Tabelle

dann Spalte

appointments

		0	1	2	3	4	5	6	...
zuerst Zeile	0	"Zzzz"	""	""	""	""	""	""	"ring"
	1	""	""	""	""	""	""	""	""
	2	""	""	""	""	""	""	""	""
	3	""	""	""	""	""	""	""	""
	4	""	""	""	""	""	""	""	""
	5	""	""	""	""	""	""	""	""
	6	""	""	""	""	""	""	""	""
	7	""	""	""	""	""	""	""	""
	8	""	"Party"	""	""	""	""	""	""
	:								

- appointments[0][0] == "Zzzz";
- appointments[0][1] == "";
- appointments[0][6] == "ring"
- appointments[8][1] = "Party"

Kalender erweitern

- Lösung: Array von Arrays (**mehrdimensionales** Array)

- Array mit 31 Elementen des Typs
Array mit 24 Elementen des Typs
String

- Deklaration der Variable:

```
String[][] appointments;
```

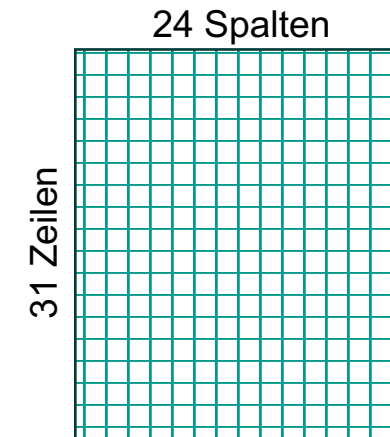
- Erzeugung des Arrays:

```
appointments = new String[31][24];
```

- oder in einem Schritt:

```
String appointments = new String[31][24];
```

- Wichtig: Dieser Kalender ist ein "rechteckiges" Array (jede Zeile hat gleich viele Spalten), daher kann man es in einem Schritt erzeugen



code11.
Multidimensi
onalArrays1

Zugriff auf Elemente

- Index von außen nach innen
- `calendar[day]` ist die day-te Komponente des äußeren Arrays. `calendar[day]` ist selbst ein Array von 24 Strings.
- `calendar.length` gibt die Anzahl der Elemente im äußeren Array an (hier: 31 Tage).
- `calendar[day].length` gibt die Anzahl der Elemente im inneren Array an (hier: 24 Stunden).
- `calendar[day][hour]` ist der Eintrag für den day-ten Tag zur hour-ten Stunde

```
appointments[2][8] = "3. Tag, 8 Uhr-Termin: Vorlesung";
```



code11.
Multidimensi
onalArrays1

calendar im Speicher

Code

```
String[][] appointments;  
// ...
```

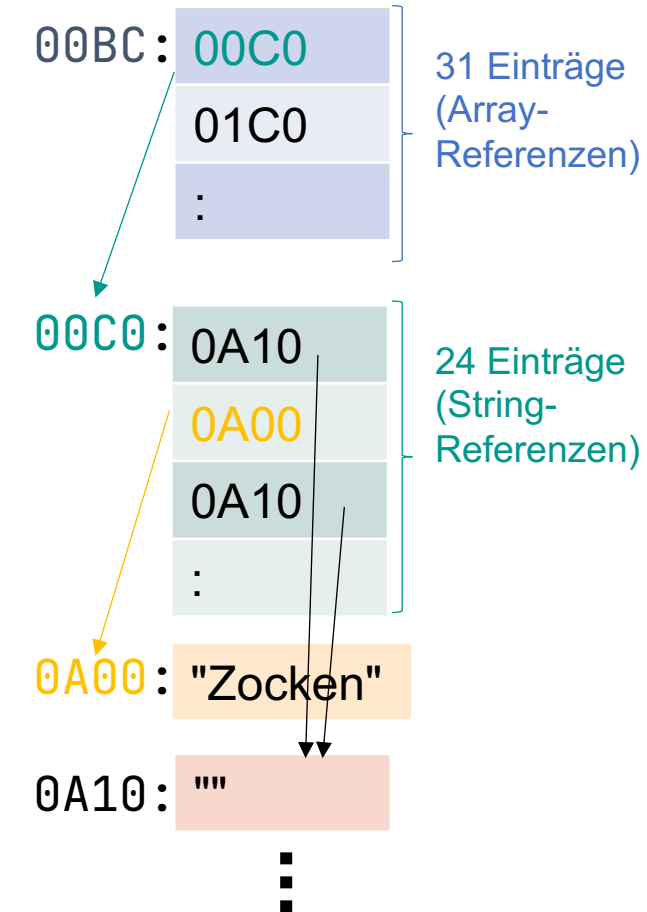
Zugriff auf appointments[0][1]:

- im Stack unter appointments nachsehen: Dort steht die Speicheradresse 00BC (Referenz auf Array)
- 0. Eintrag im Heap unter Adresse 00BC nachsehen: Speicheradresse 00C0 (Referenz auf Array)
- 1. Eintrag in Array unter Adresse 00C0 nachsehen: Dort steht "Zocken"
- Also ist appointments[0][1] = "Zocken"

Stack Memory

appointments: 00BC

Heap Space



Erweiterung Kalender (Live-Coding)

- Wir ersetzen im SimpleCalendar das eindimensionale Array appointments durch ein mehrdimensionales Array, mit 31 Zeilen (Tage) und 24 Spalten (Stunden)
- Fahrplan:
 - Array einführen
 - Array-Variable mit leeren Strings initialisieren
 - Termine einlesen (zusätzliche Eingabe des Tages)
 - Ausgabe der Termine (zusätzliche Eingabe des Tages)
 - Initialisierung mit "" für Ausgabe ohne null
- Das alles folgt (zur Nachbereitung) auch noch mal auf den folgenden Folien als Einzelschritte und als fertige Version in MultidimensionalCalendar.

Kalender erweitern: Array anlegen und initialisieren

```
String appointments[][] = new String[31][24];

// Vorbereitung für die Ausgabe.
// Wenn wir das nicht machen, steht überall der Wert null
for (int day = 0; day < appointments.length; day++) {
    for (int hour = 0; hour < appointments[day].length; hour++) {
        appointments[day][hour] = "";
    }
}
```



code11.
MultiDay
Calendar

Kalender erweitern: Eingabe des Tags

```

switch (choice) {
    case 1 -> { // Neuer Eintrag
        System.out.print("Welcher Tag? (1-31) ");
        int inputDay = scanner.nextInt();
        System.out.print("Wieviel Uhr? (0-23) ");
        int inputHour = scanner.nextInt();
        System.out.print("Termin? ");
        appointments[inputDay - 1][inputHour] = scanner.next();
    }
    case 2 -> { // Termine ausgeben
        System.out.println("Welcher Tag? ");
        int outputDay = scanner.nextInt();
        for (int hour = 0; hour < appointments[outputDay - 1].length; hour++) {
            System.out.format("%2dh: %s\n", hour, appointments[outputDay - 1][hour]);
        }
        System.out.println("-----");
    }
    case 3 -> // Beenden
        done = true;
    default -> // Falsche Eingabe
        System.out.println("Eingabefehler");
}

```



code11.
MultiDay
Calendar

Uneinheitliche, mehrdimensionale Arrays

```
String[][] monthsAndDays = new String[12][];  
monthsAndDays[0] = new String[31];  
monthsAndDays[1] = new String[28];  
monthsAndDays[2] = new String[31];  
monthsAndDays[3] = new String[30];  
:
```

Beispiel (nicht "rechteckig"):

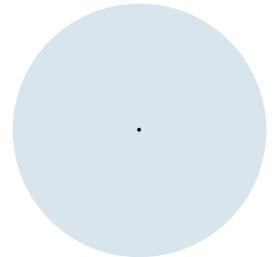
- Ebene 1: Monate, Ebene 2: Tage
- Januar hat 31 Tage, Februar hat 28 (oder 29) Tage, ...

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Jan, 0																															
Feb, 1																															
Mär, 2																															
Apr, 3																															
Mai, 4																															
Jun, 5																															
Jul, 6																															
Aug, 7																															
Sep, 8																															
Okt, 9																															
Nov, 10																															
Dez, 11																															



code11.
Multidimensi
onalArrays2

Arraykopie (mehrdimensional)



Kopieren mehrdimensionaler Arrays

Situation:

- Wir erhalten ein multidimensionales Array.
- Wir möchten dieses Array kopieren.
- Anschließend möchten wir mit der Kopie arbeiten, dort Werte verändern (im Beispiel Termine ändern), ohne dass das Original verändert wird.

Kopieren mehrdimensionaler Arrays

- Beispiel Monatskalender

```
String[][] oldCalendar = new String[31][24];  
oldCalendar[4 - 1][14] = "Vorlesung"; // 4. Tag, 14 Uhr  
  
String[][] newCalendar = ???; // Kopie erstellen  
  
newCalendar[4 - 1][14] = "Seminar";  
  
// erwartete Ausgabe: Vorlesung  
System.out.println(oldCalendar[4 - 1][14]);  
  
// erwartete Ausgabe: Seminar  
System.out.println(newCalendar[4 - 1][14]);
```



code11.
Multidimensi
onalCopy0

Kopieren mehrdimensionaler Arrays

- 1. Versuch (nur Referenz)

```
String[][] newCalendar = oldCalendar;
```

- Vorsicht ist geboten!



code11.
Multidimensi
onalCopy1

oldCalendar und newCalendar im Speicher

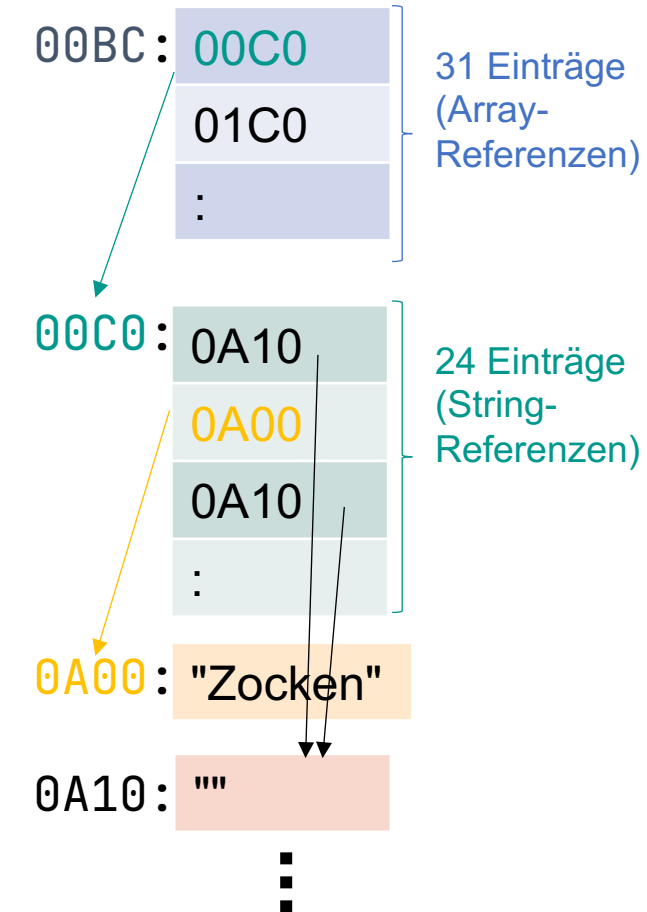
Code

```
String[][] oldCalendar;  
String[][] newCalendar  
    = oldCalendar;
```

Stack Memory

oldCalendar: 00BC
newCalendar: 00BC

Heap Space



Vorsicht: Egal, ob man über oldCalendar oder newCalendar einen Eintrag ändert: Änderungen wirken sich immer auf denselben Speicherbereich aus, da hier keine Kopie der Speicherinhalte stattgefunden hat.

Kopieren mehrdimensionaler Arrays

- 2. Versuch (flache Kopie)

```
String[][] newCalendar;  
newCalendar = oldCalendar.clone();
```

- Vorsicht ist geboten!



code11.
Multidimensi
onalCopy2

oldCalendar und newCalendar im Speicher

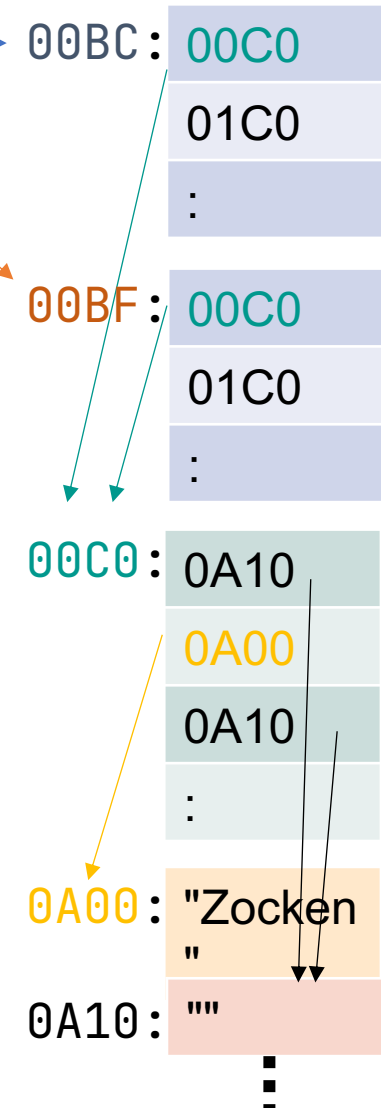
Code

```
String[][] oldCalendar;  
String[][] newCalendar  
    = oldCalendar.clone();
```

Stack Memory

oldCalendar: 00BC
newCalendar: 00BF

Heap Space

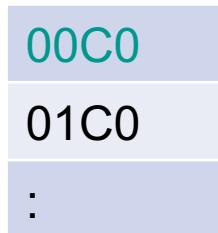


oldCalendar und newCalendar im Speicher

Code

```
String[][] oldCalendar;  
String[][] newCalendar  
    = oldCalendar.clone();
```

Vorsicht: `.clone()` erzeugt nur eine flache Kopie des Arrays A der oberen Ebene, welches Referenzen enthält:

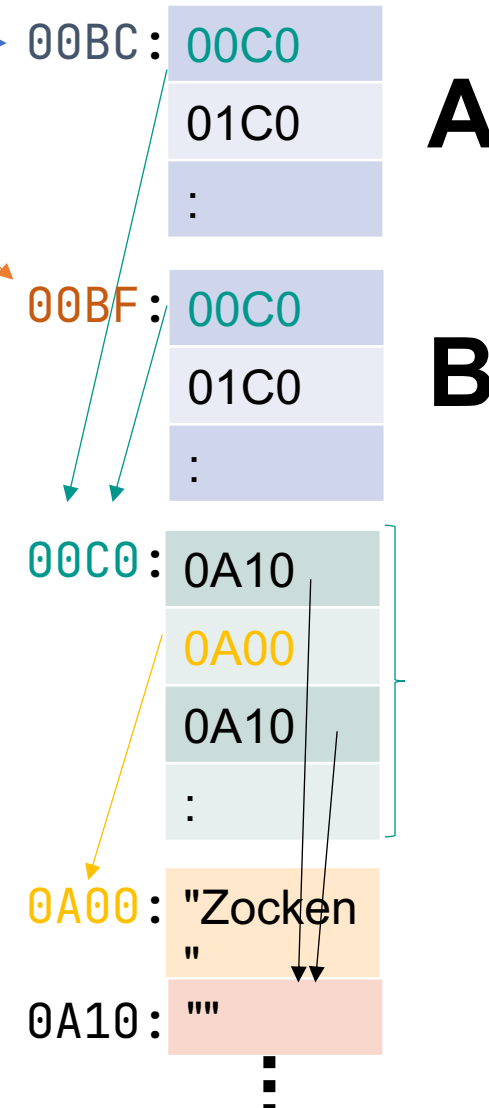


Nur die Referenzen werden dabei kopiert. Sie zeigen aber in beiden Arrays A und B auf die gleichen Speicherinhalte.

Stack Memory

oldCalendar: 00BC
newCalendar: 00BF

Heap Space



Kopieren mehrdimensionaler Arrays

- 2. Versuch (flache Kopie)
- Hinweis: Auch die folgenden Methoden erzeugen nur flache Kopien und führen so zum gleichen Ergebnis:

```
String[][] newCalendar = new String[31][24];  
System.arraycopy(oldCalendar, 0, newCalendar, 0, oldCalendar.length);
```

```
String[][] newCalendar = Arrays.copyOf(oldCalendar, oldCalendar.length);
```

```
String[][] newCalendar = Arrays.copyOfRange(oldCalendar,  
                                             0, oldCalendar.length);
```



code11.
Multidimensi
onalCopy2

Kopieren mehrdimensionaler Arrays

- 3. Versuch (tiefe Kopie mit Schleife)

```
String[][] newCalendar = new String[31][24];  
for (int day = 0; day < newCalendar.length; day++) {  
    for (int hour = 0; hour < newCalendar[day].length; hour++) {  
        newCalendar[day][hour] = oldCalendar[day][hour];  
    }  
}
```

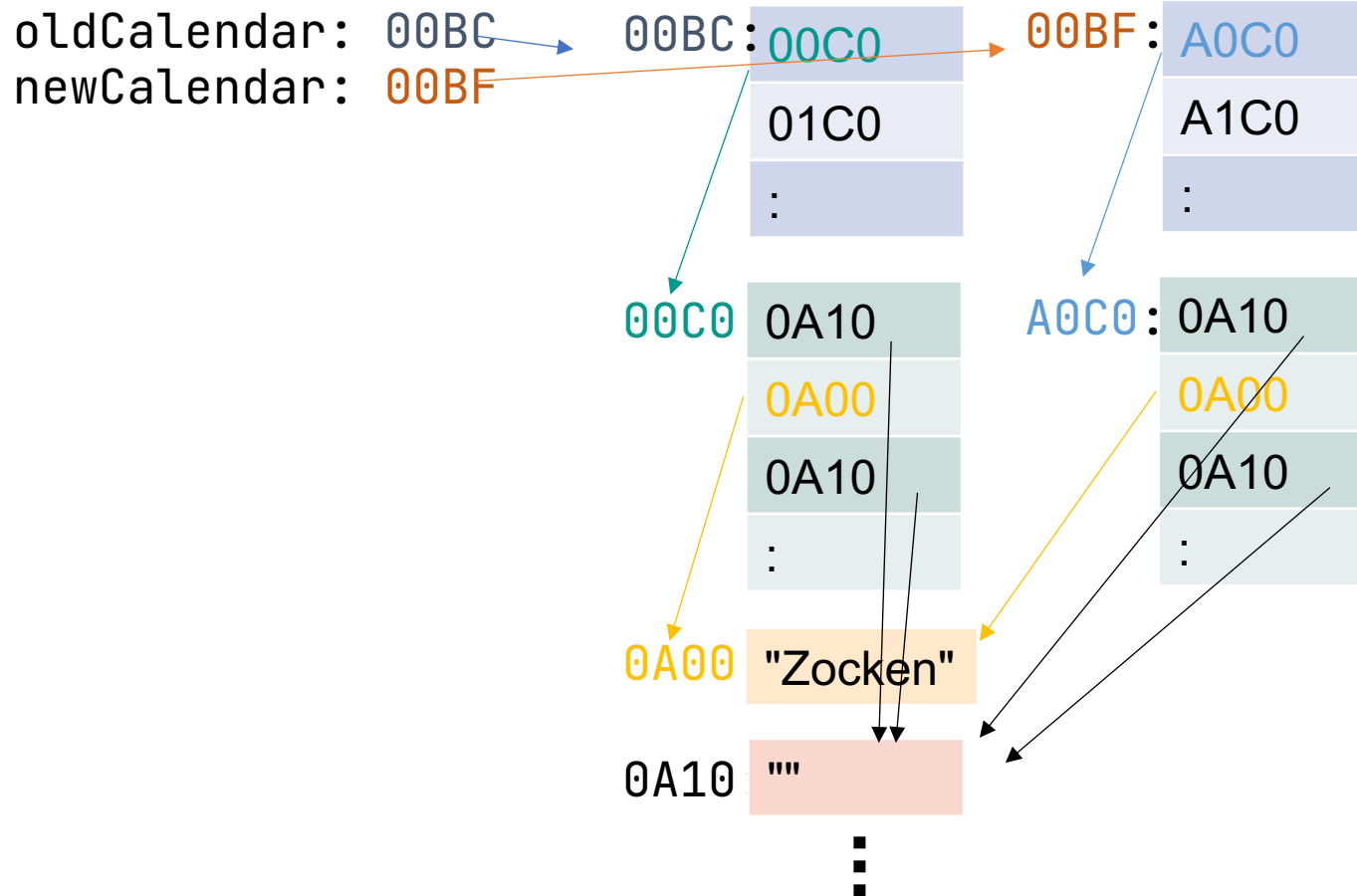


code11.
Multidimensi
onalCopy3

oldCalendar und newCalendar im Speicher

Stack Memory

Heap Space

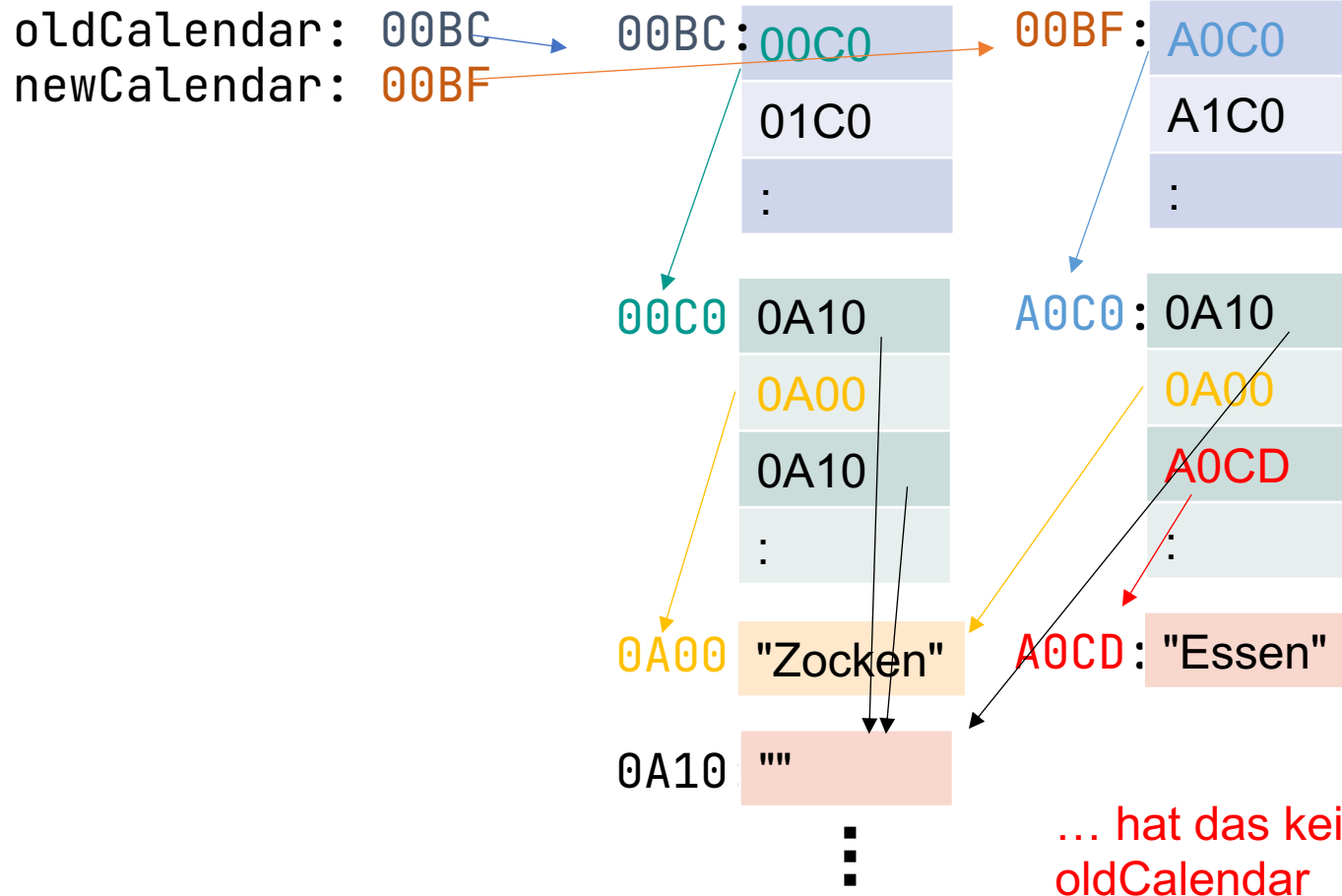


← Wenn wir jetzt diesen Eintrag in `newCalendar` ändern...

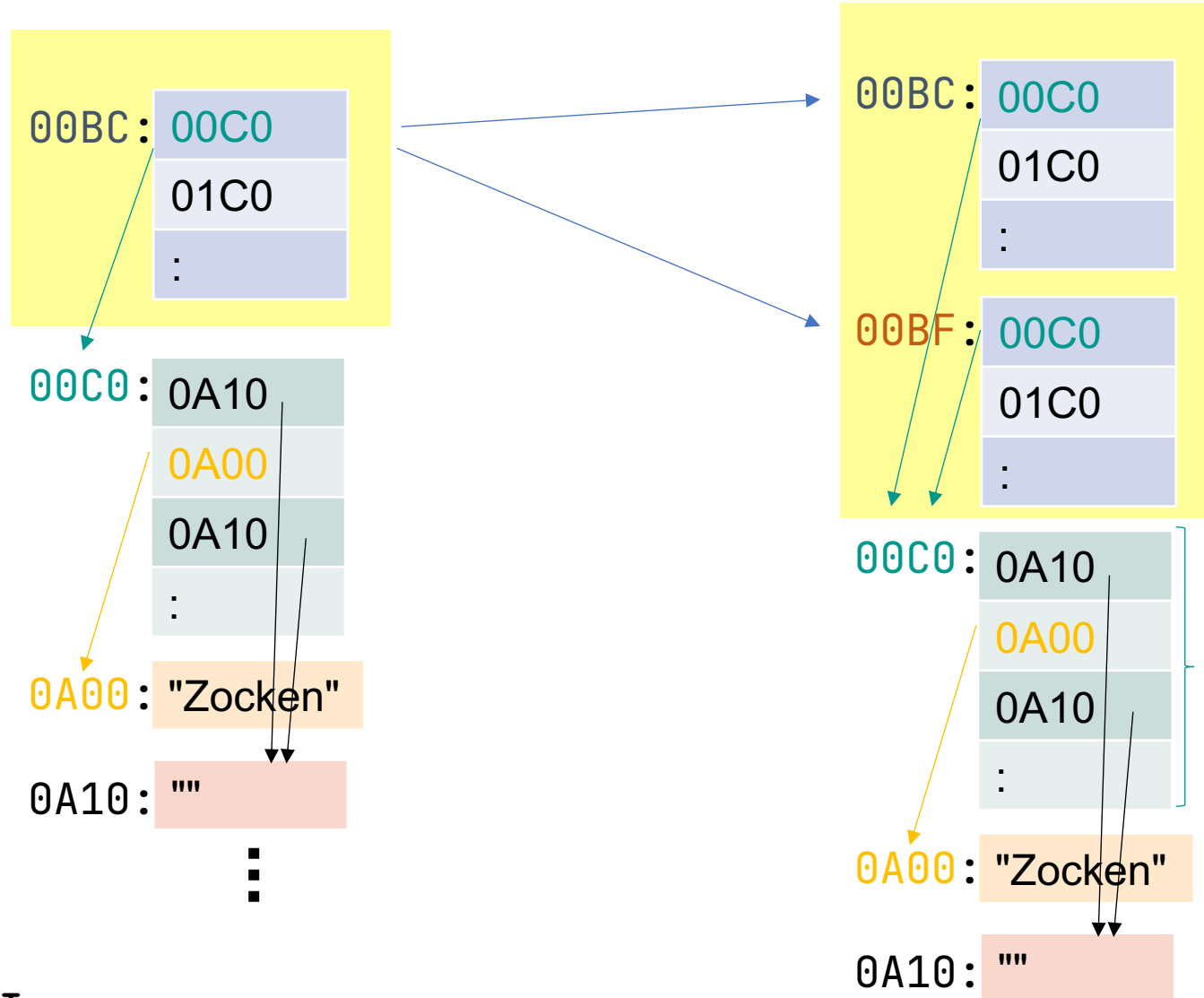
oldCalendar und newCalendar im Speicher

Stack Memory

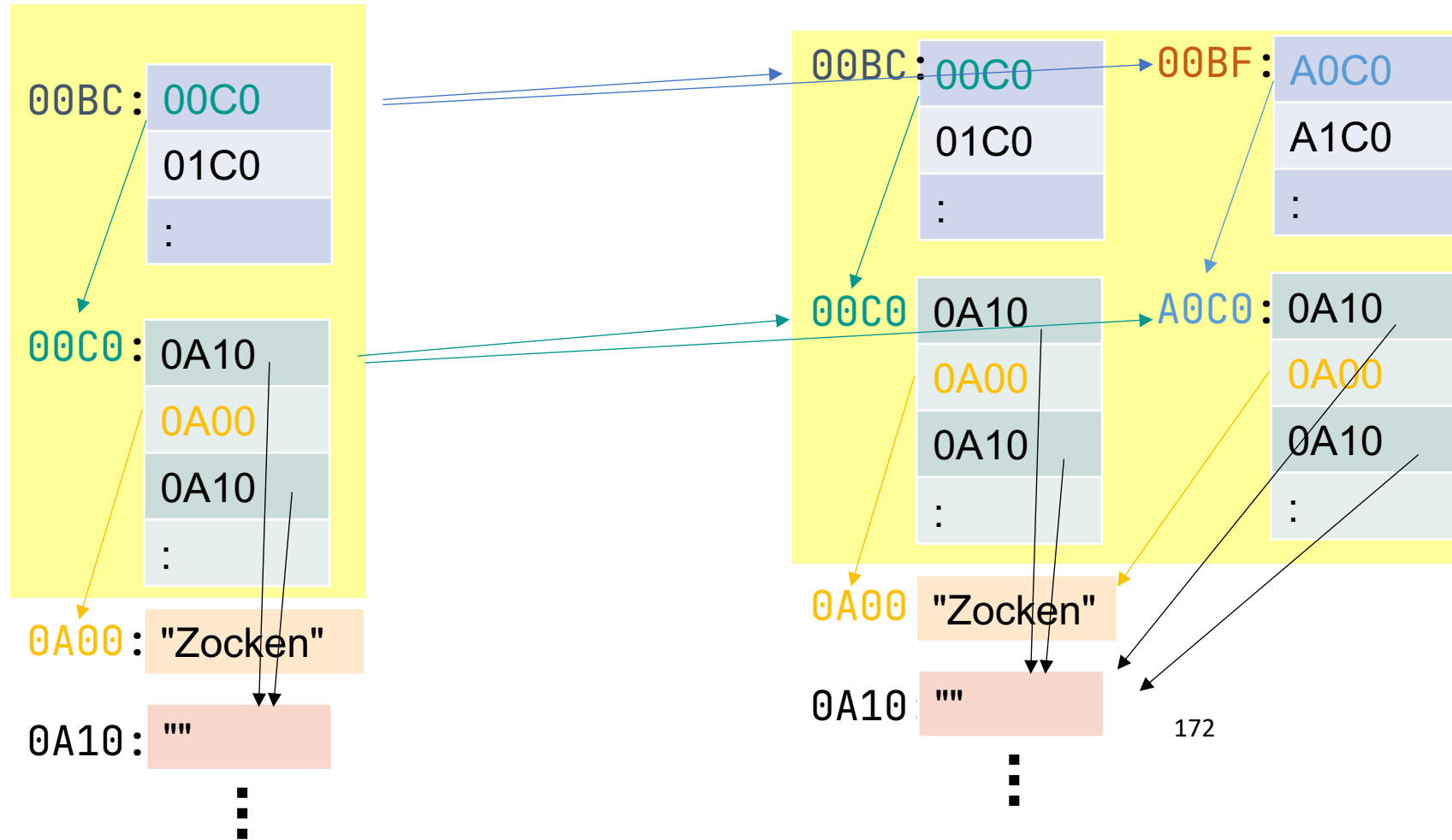
Heap Space



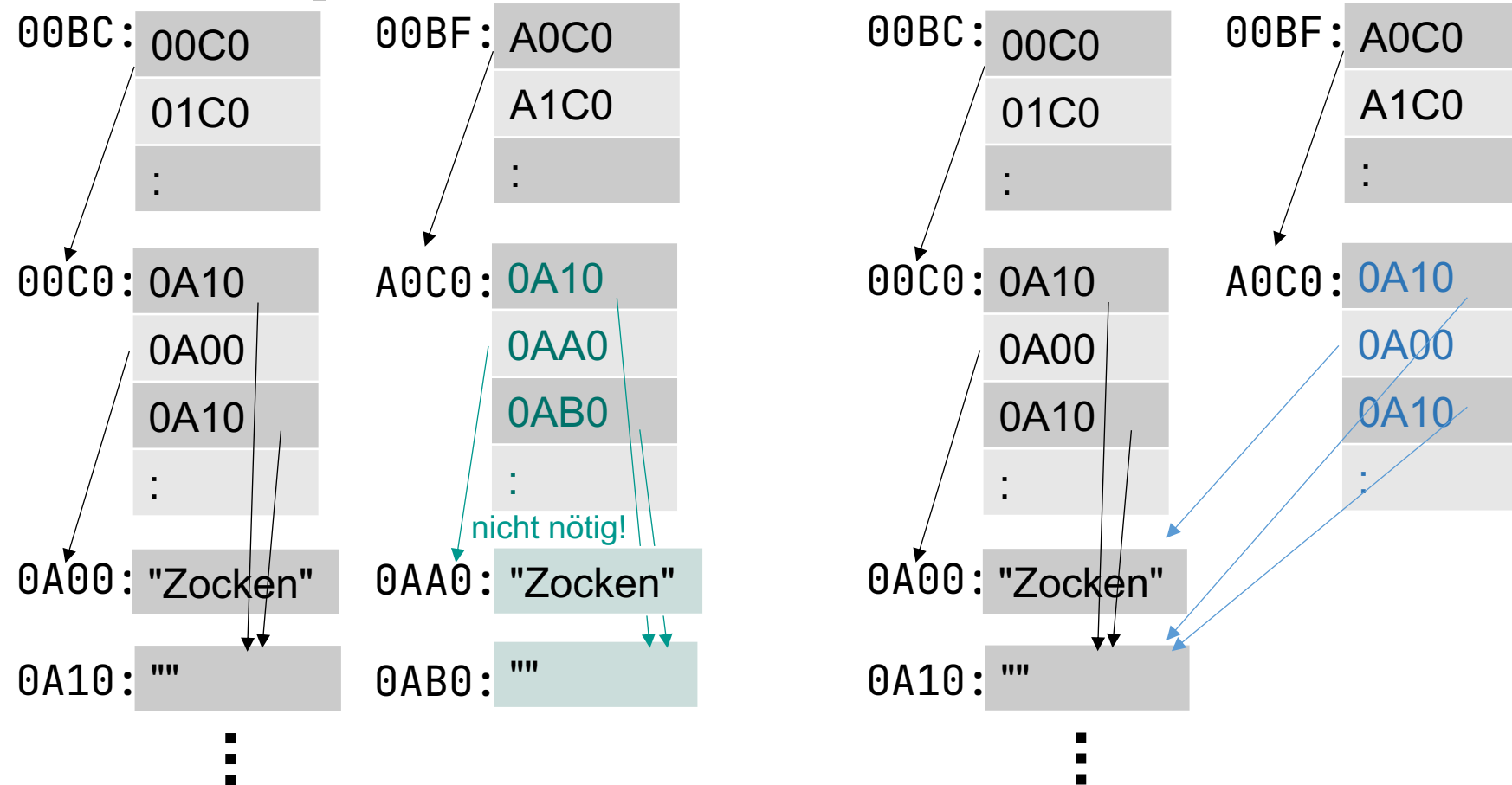
Flache Kopie: nur erste Ebene kopiert



Tiefe Kopie: Kopie aller Array-Ebenen



Tiefe Kopie: Noch tiefer?



Strenggenommen müsste eine tiefe Kopie auch **die einzelnen String-Einträge kopieren (links)**. Da man einen String-Inhalt im Speicher jedoch nachträglich nicht mehr ändern kann (Stichwort *immutable objects*), reicht hier **eine Kopie der Referenzen (rechts)**.